Lesson for the course “Human-Computer Interaction”

Time, space, and eye-hand coordination

Prof. Luca Turchet

UNIVERSITY
OF TRENTO - Italy

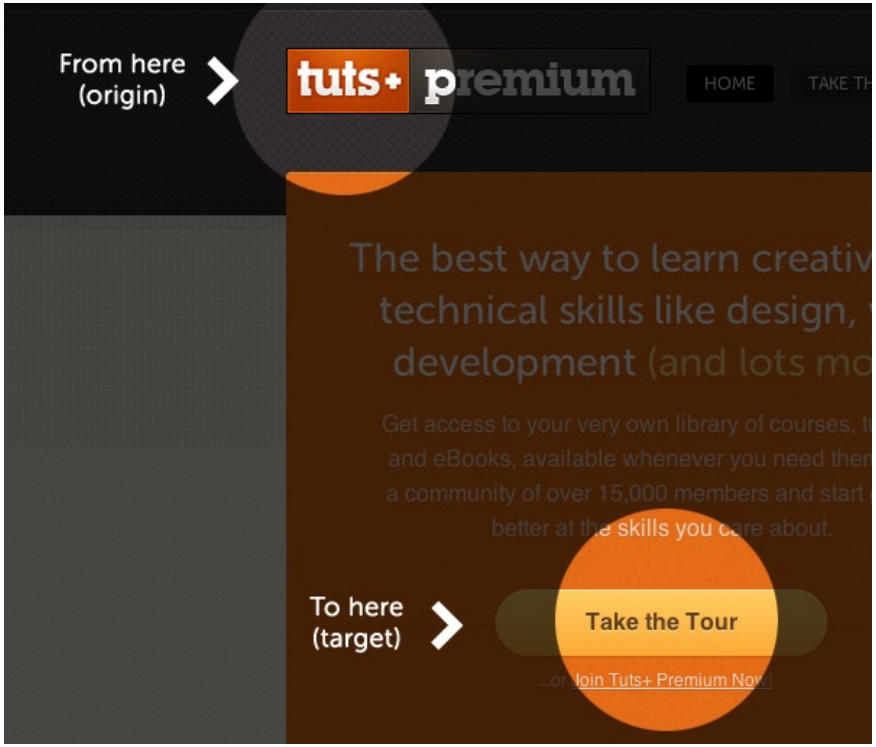
Laws for pointing devices

- **Fitts' law**
- **Steering law**
- Two laws that govern human pointing behavior for all known types of pointing devices, including fingers on touchscreens
- We discuss the implications of these two laws for user-interface design
- We also discuss time requirements for interactive systems

Fitts' law

- It predicts that the **time** required to move to a target area is a function of the ratio between the distance to the target and the width of the target

- **It is used to model the act of pointing,** either by physically touching an object with a hand or finger, or virtually, by pointing to an object on a computer monitor using a pointing device



Fitts' law

$$T = a + b \log_2 \left(\frac{D}{W} + 1 \right)$$

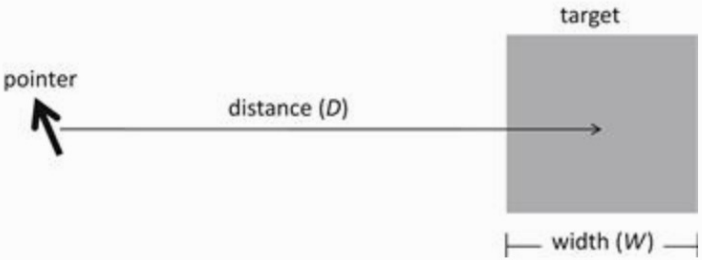
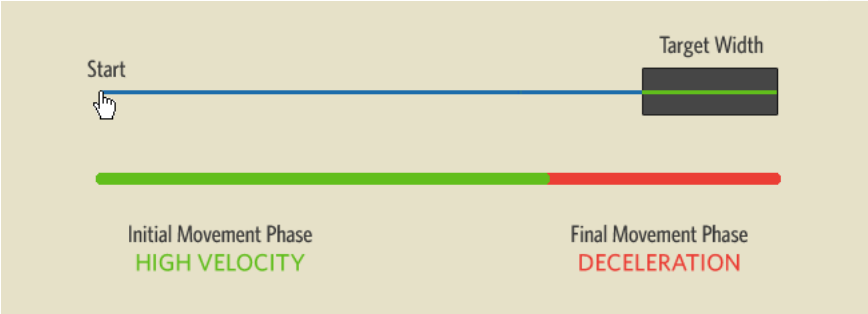


FIGURE 13.1 Fitts' law: pointing time depends on distance (*D*) and the width (*W*) of the target.

- *T* is the average time to complete the movement to the target
- *D* is the distance from the starting point to the center of the target
- W is the width of the target along the axis of movement of the pointer
- *a* and *b* are constants that depend on the choice of input device and are usually determined empirically
- From the formula, you can see that **as the distance (D) increases, the time (T) to reach the target increases, and as the target width (W) increases, the time to reach it decreases**

Velocity over time

- Movement toward the target starts slowly, but rapidly accelerates until reaching some maximum speed
- This initial movement is fairly gross: it is essentially a shot in the general direction of the target, without much control
- As the pointer nears the target, the movement speed slows as the person's hand–eye feedback loops take control
- The movement ends slowly, with finer and finer corrections, until the pointer is on the target



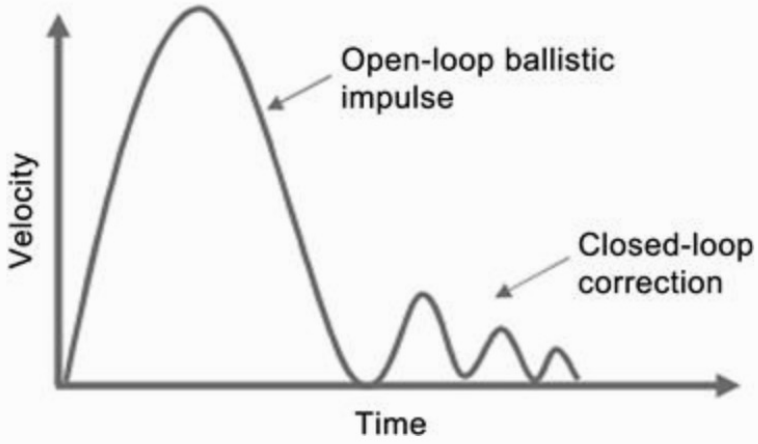


FIGURE 13.2 Graph of velocity over time as pointer moves to target.

Time vs target size, time vs distance

- The more the distance decreases or the target grows, the less the decrease in pointing time
  - If a target is tiny and you double its size, the time people take to hit it decreases, but if you double the size again, the pointing time doesn't improve quite as much
  - **So beyond a certain size, making a target even larger provides little added benefit, and below a certain distance, moving a target even closer doesn't help much**

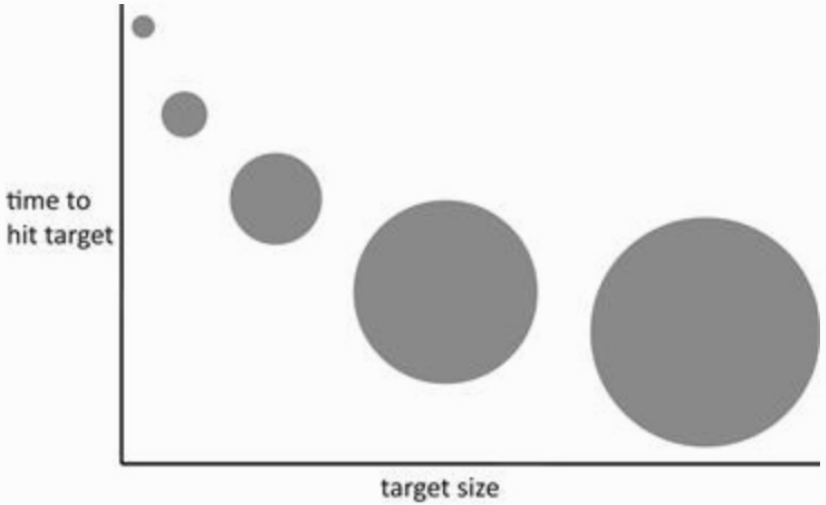


FIGURE 13.3 Diminishing marginal benefit of increasing click-target size (if *D* is constant).

Implications of Fitts' law for UI design

- The size of a target and its distance from the user's current position within the user **interface affect the user experience in a number of ways**
- Various implications of the law for UI design
  - Both for desktop and mobile devices
- Guidelines

The *larger* a target, the faster it is to acquire

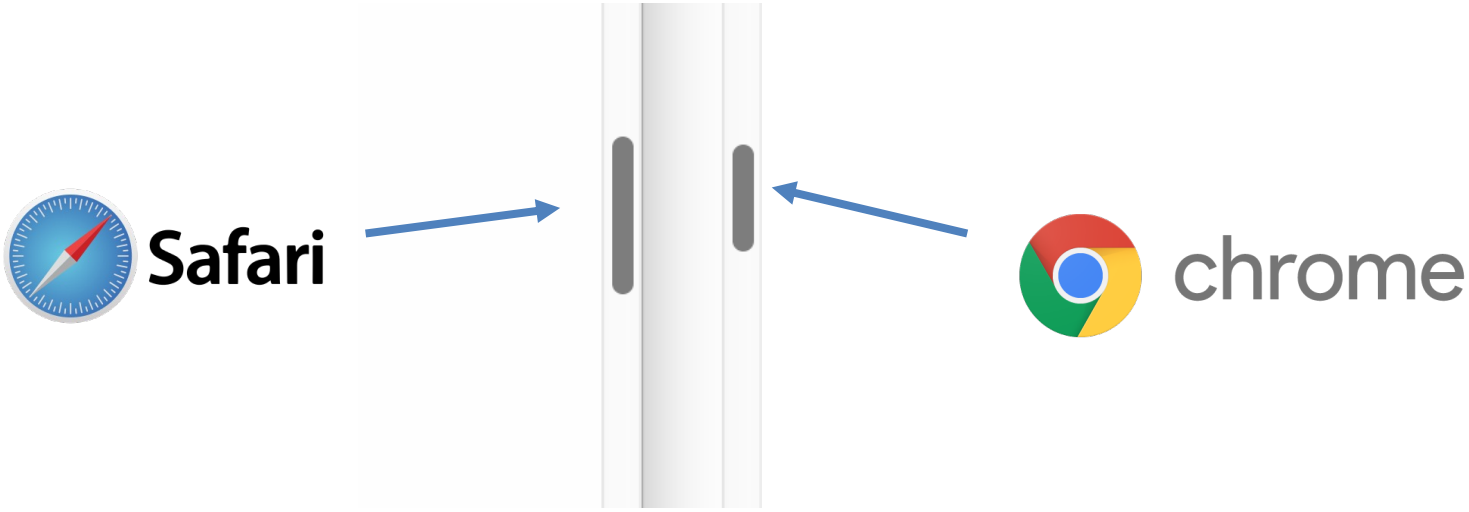
- It is important to ensure that users are able to click anywhere on an interactive element to carry out the assigned action
  - E.g., links (whether they are in the form of text, images or buttons) should afford clicking **on the whole region**
- Unfortunately, in web design especially, users are often forced to position the cursor directly over a specific portion of a link in order to interact



Note: in Axure you can use hot spots for this purpose

The *larger* a target, the faster it is to acquire

- Comparison between size of the handle of the scrollbar for Apple's Safari and Google's Chrome
  - Same URL and same position in the page



  - In terms of scrolling, Safari is more usable than Chrome since the handle is easier to reach given the fact that it covers a larger area

The *larger* a target, the faster it is to acquire

- Increasing the size of a button to make it easier to click is also a popular technique among designers for communicating priorities within the interface and prompting or calling users to perform a particular action

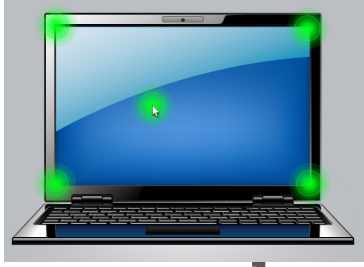


- **…but bigger is not always better**
  - Recall that if you make a small button e.g., 10% larger, it will become much more clickable, but if you make an already very large button 10% larger, you will not gain much more in terms of its usability
  - **Buttons that are large enough to demand attention without disrupting the visual balance of your page are what maximize usability**

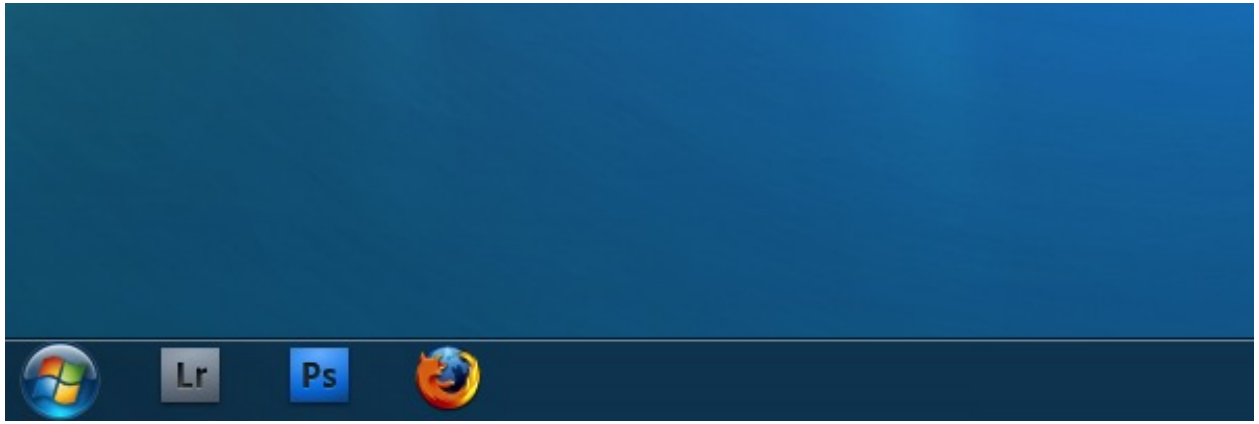
Edges

- A noteworthy prediction of Fitts' law is that, if the pointer or finger is blocked from moving past the edge of the screen, targets at the edge will be very easy to hit:
  - People can just yank the pointer toward the target until the edge stops it, with no need for slow, fine adjustments at the end of the movement
- Thus, from the point of view of Fitts' law, **targets at the screen's edge behave as if they were much larger than they actually are**
- However, this edge-pointing detail of the law applies mainly to desktop and laptop computers, because modern smartphones and tablet computers don't have raised edges that physically stop fingers

Edges: corners

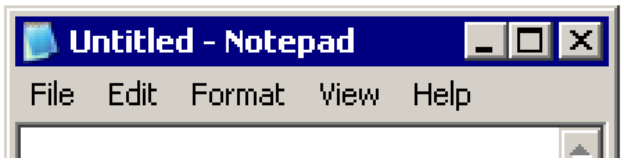


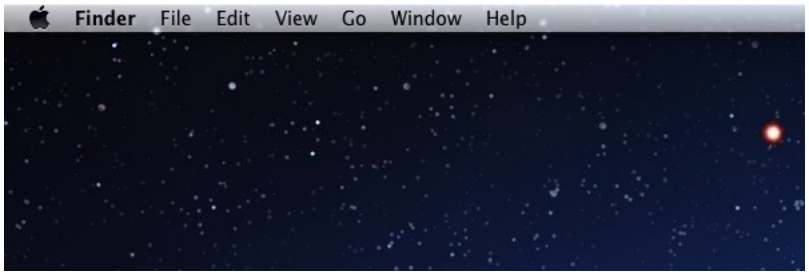
- Fitts' law indicates that **the one of the most quickly accessed targets on any computer display are the four corners of the screen**
- As the mouse cursor stops at the edge of the screen, corners can be considered to have an "infinite" width
- The user needs much less precision because they can simply fling the mouse in the direction of a corner and the limitations of the screen restrict where the pointer ends up
- This is partly why you see the windows start menu and the Apple menu in the corners of your screen



Edges: top and bottom

- Similarly, the top and bottom are easier to access due to screen limitations
- These are not as easy as corners because they are only limited vertically, but still allow for quicker access than a point in the middle of the screen
- This is why Apple always places application menus at the top of the screen instead of at the top of the application window





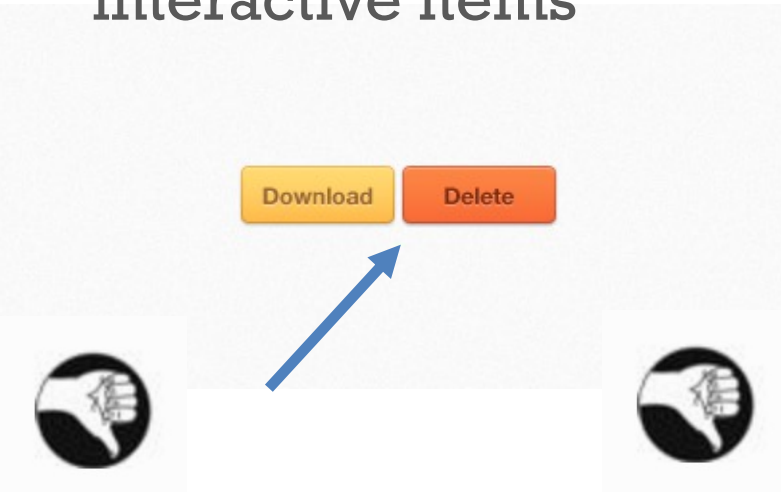
- Apple's menus are faster to navigate than Windows' menus

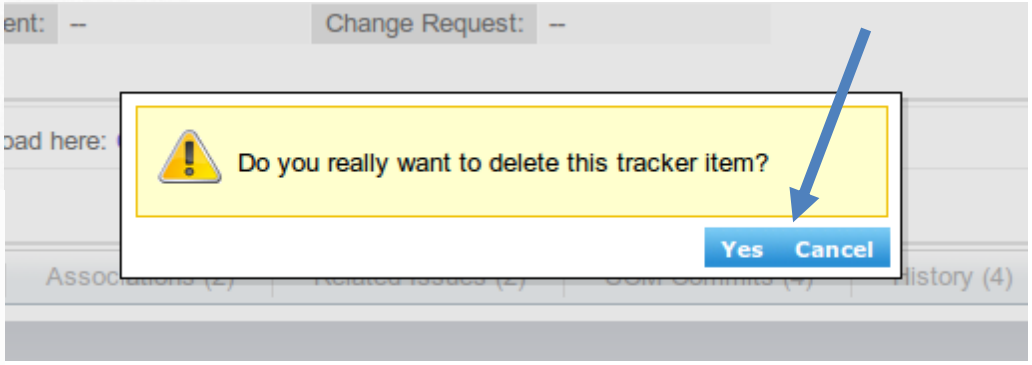
The *closer* a target, the faster it is to acquire

- Fitts's law also suggests that you should decrease the distance between objects that users use in a logical order
  - For instance, the login button is always very close to the username and password form fields
- When certain buttons are **related**, you should organize them in a way that's **easy to remember and find**
  - Grouping similar buttons creates a familiar mental map on your website/app that users can basically memorize
- If you place next to each other the links and buttons that users are most likely to access **on a regular basis**, rather than distribute them across the interface, you will speed up interaction by reducing the number of pixels the cursor will have to travel

The *closer* a target, the faster it is to acquire

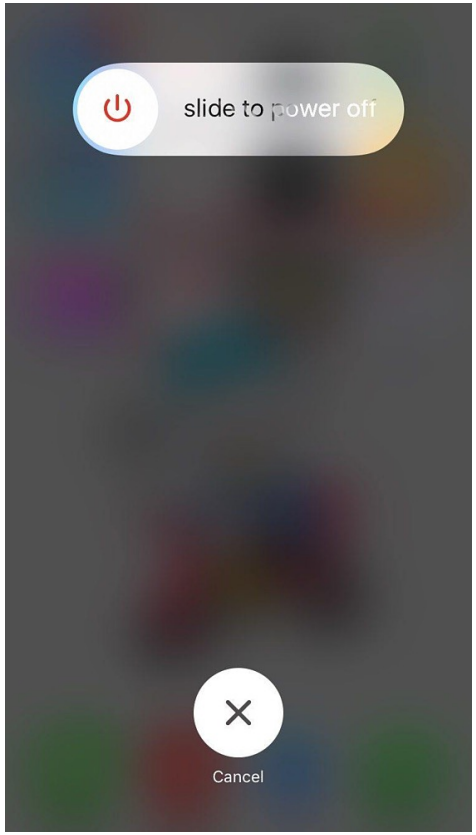
- Fitts's Law suggests placing elements directly next to each other to **minimize cursor movement**, which would also save some space
- However, saving those few pixels can result in users accidentally clicking the wrong item, especially when item boundaries are not easily discernible or when focused items are not highlighted distinctly enough
- It is important to account for high risk interactive elements that you don't want the user to accidentally activate: those should often be kept **further away** from heavily used interactive items





Less effort isn't always better

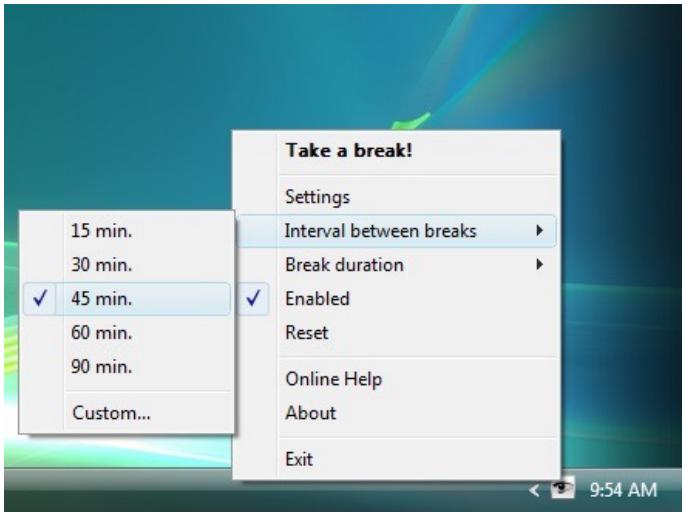
- Some crucial commands that are easy to do, like turning off your phone, don't allow users to minimize the costs of their mistakes
  - Once they trigger them, users can't undo them



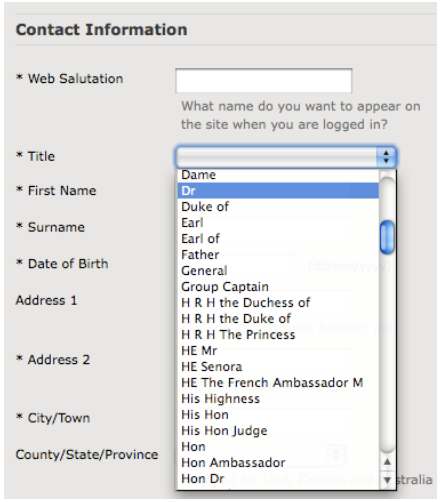
- To prevent frequent **accidental** shutdowns on their phones, Apple makes users swipe a slider to turn off their phone
- And to rescind a power off, all users have to do is press the cancel button
- Turning the phone off requires swiping (complex action), cancelling the action requires to press (simple action: it may occur accidentally and the consequences are different from turning off)

The *closer* a target, the faster it is to acquire: Menus

- The fastest-to-acquire pixel in any situation is simply the pixel at the current cursor position
- **Pop-up menus** better support immediate selection of interactive elements than dropdown menus as the user does not have to move the cursor from its current position (mouse right-click)
- Therefore, graphical designs that allow the user to interact without moving help to reduce the "travel time"



- Pop-up menu



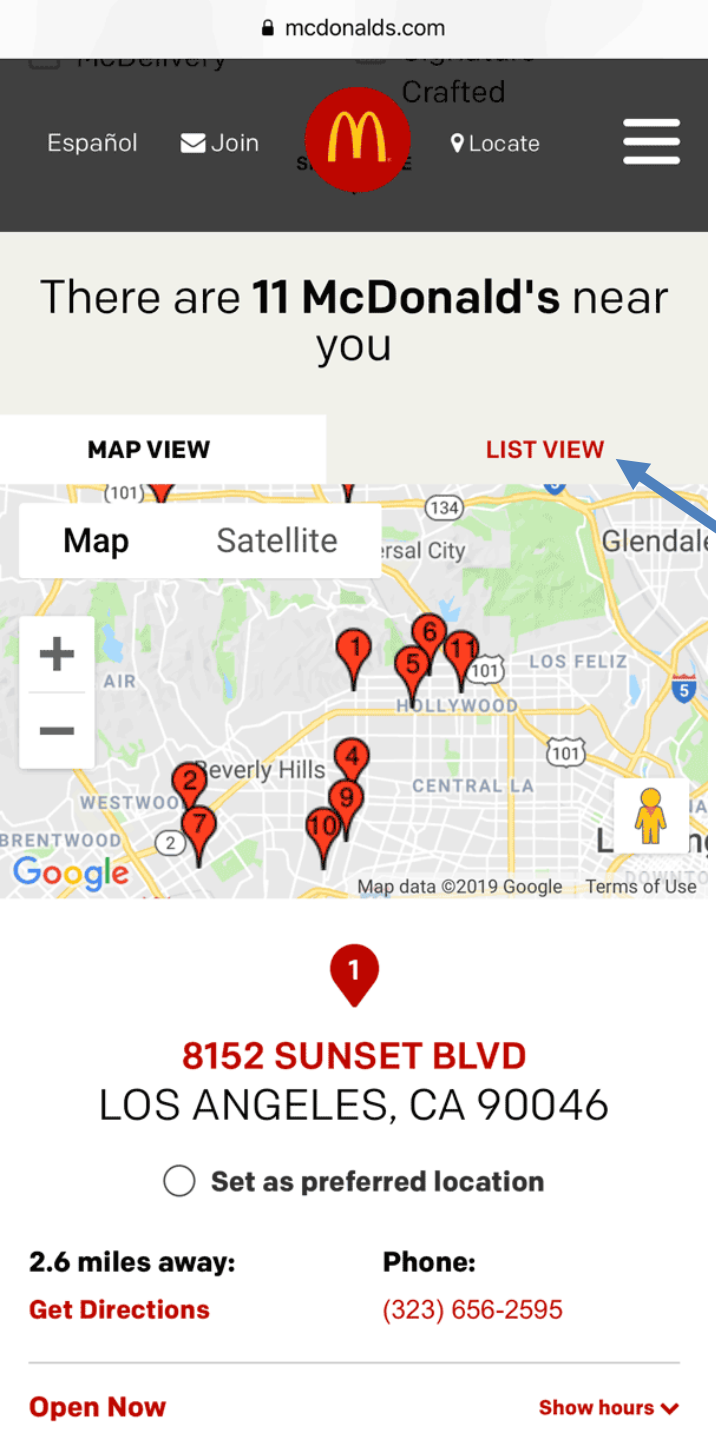
- Dropdown menu

Touch targets on touchscreens

- Handheld devices can generally be categorized into phones and tablets
  - These devices can be used in two orientations, vertical and horizontal
  - The thumb or fingers act as interaction tool in mobile user experience
- **According to Fitts' law when touch targets are too small, users take longer to tap them**
- Interactive elements must be wide enough to support adequate selection time and prevent *fat-finger errors*
  - "fat finger" problem: users' fingers are larger than the desired targets
  - Designers who prioritize aesthetics over functionally too often forget to create targets that can be easily and accurately selected

Example

- Maps on touchscreens often cause touch-target errors
  - When many locations are presented in a map view, the markers for each location are so small and densely packed that it becomes nearly impossible to precisely select a particular pinpoint



McDonalds' mobile site:

The locator search-results page presented a map with markers too small and close together

Information about a location was displayed by tapping the corresponding pinpoint

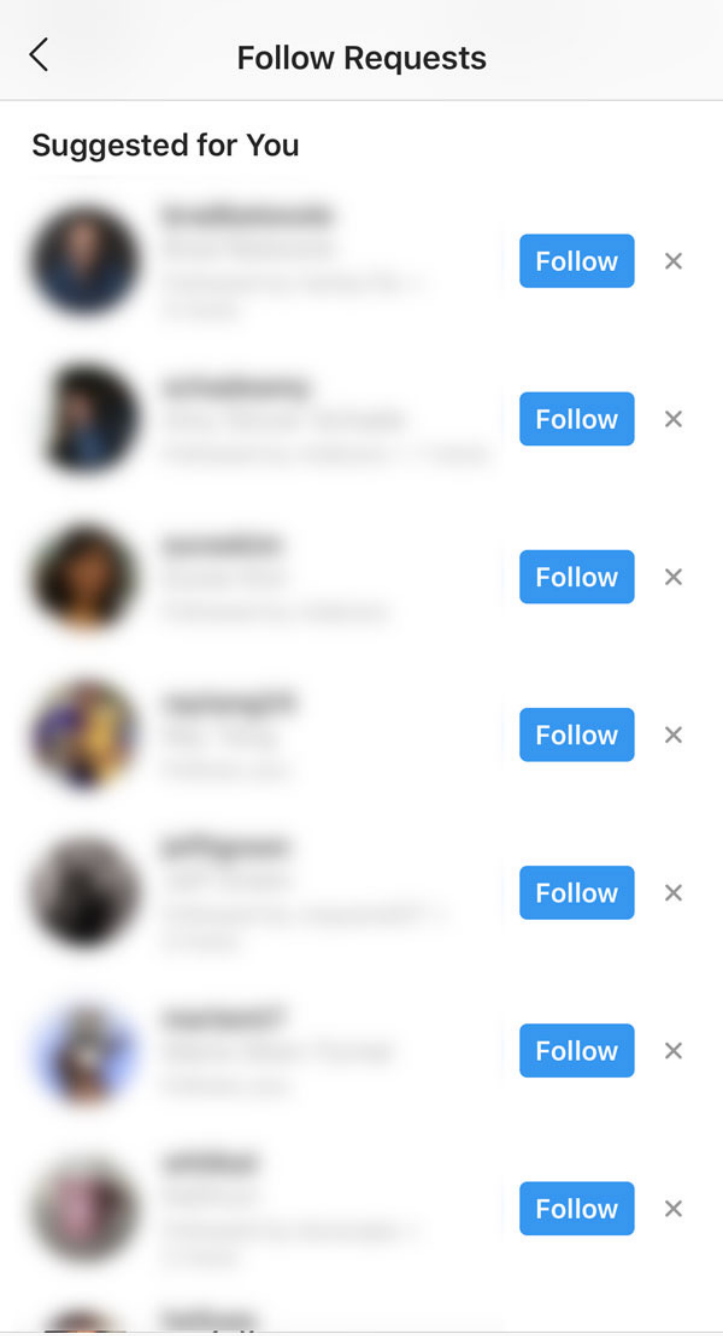
Thankfully, a **list view** of locations was also available

Also, + and – widgets allow users to zoom

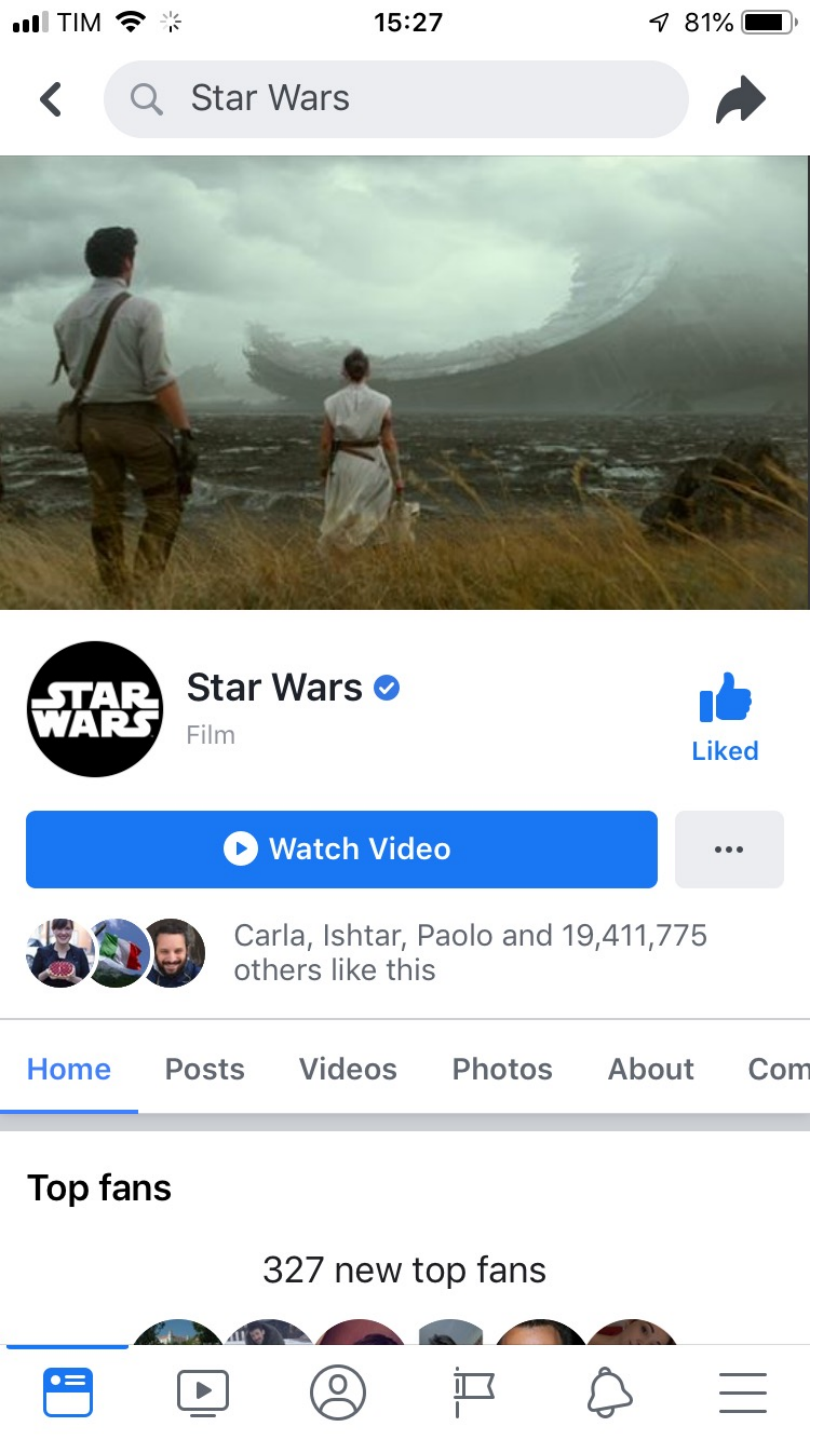
Touch targets on touchscreens

- Touch-target errors can happen because the spacing between the elements is too small
- More space between them would prevent users from tapping the wrong one
- Of course, if the targets are too small, adding space between them is likely not going to help. **To avoid accidental taps, targets must first be big enough, and then also spaced well enough**

Example: Instagram for iOS

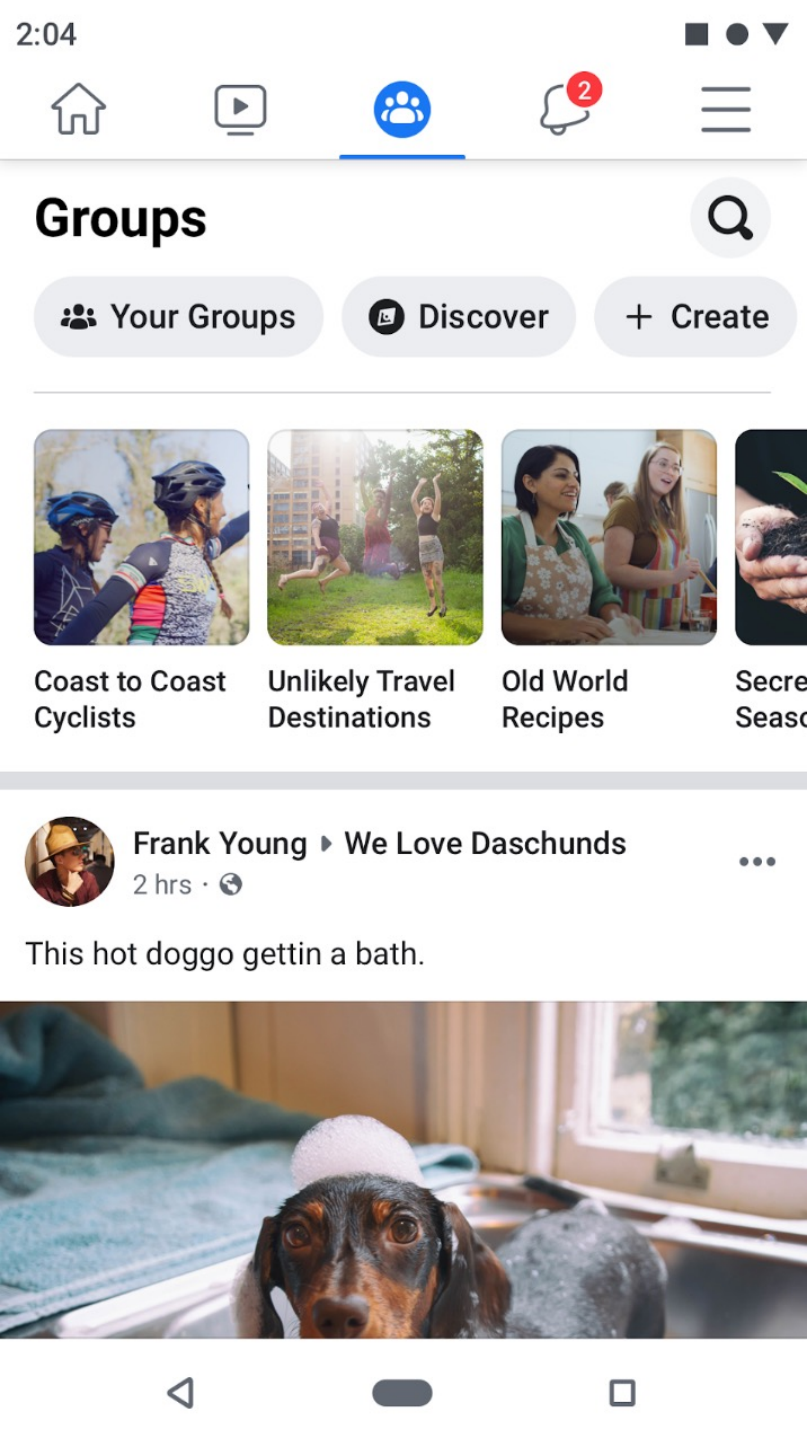


- The *x* dismiss buttons to the right of the Follow buttons are too small, and the small amount of spacing between these opposed actions is not enough to make up for their small size
- Because of this design, it's easier to simply ignore any bad suggestions rather than risk accidentally following an account when attempting to remove it from the list
- If *x* dismiss button was wider, the amount of space between the opposing actions would have been sufficient



iOS apps, vertical orientation

- The key menu buttons are placed at the bottom of the screen
- This gives users accessibility in terms of thumb movement when the device is held vertically and Fitts' law is applicable here as the time required for the action to take place is reduced to a minimum



Android apps, vertical orientation

- The key interaction buttons are placed at the top of the screen for an android device
- This causes a delay in the movement depending on the device size
- …but this is not done without due diligence: In Androids, the key system interactions buttons are placed at the bottom of the device/screen hence the menu buttons are placed at the top of the screen to avoid accidental interactions

Horizontal orientation

- In horizontal orientation, most users use both their hands to type
- The middle part of the screen becomes difficult to reach so the important buttons should be placed at the sides when the device is in landscape/horizontal orientation



Twitter app for Android users shows that the "compose tweet" interact button is placed at the bottom right of the screen, making it easier to access

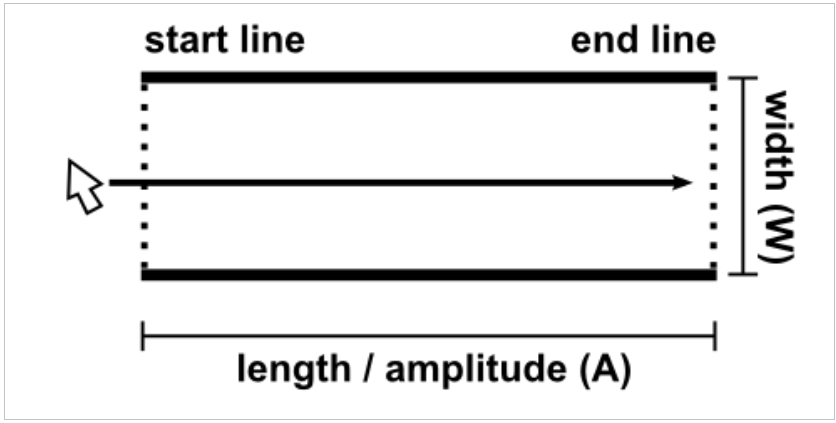
Guidelines

- **Make click-targets** (e.g., graphical buttons, menu items, links) **big enough that they are easy for people to hit**
  - Don't make people click on tiny targets
- **Checkboxes, radio buttons, and toggle switches should accept clicks on their labels** as well as on the buttons, thereby increasing the clickable area
- **Leave plenty of space between buttons/links** so people don't have trouble hitting the intended one
- **Place important targets near the edge of the screen** to make them very easy to hit
- **Display choices in pop-up menus if possible**

Steering law: moving pointers along constrained paths

$$T = a + b\left(\frac{A}{W}\right)$$

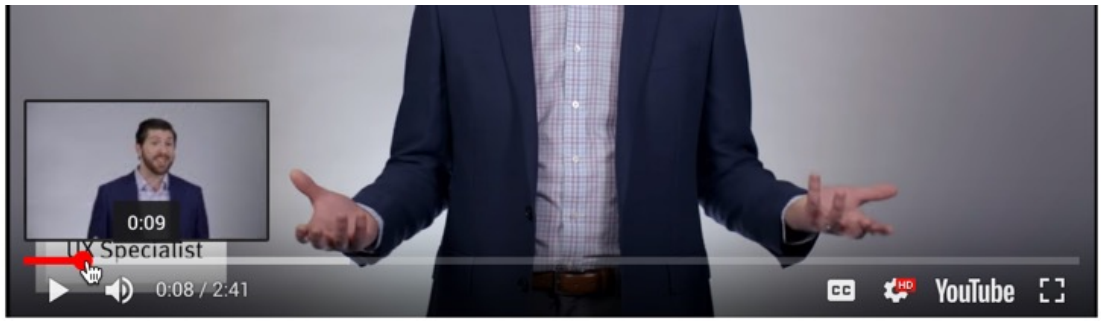
a and *b* are experimentally fitted constants



- The law says that if you must keep a pointer within a certain confined path while moving it to a target, then the wider the path, the faster you can move the pointer to the target
- Common sense: a wider path means you need not move the pointer carefully
  - Wide path: fast, easy
  - Narrow path: slow, difficult

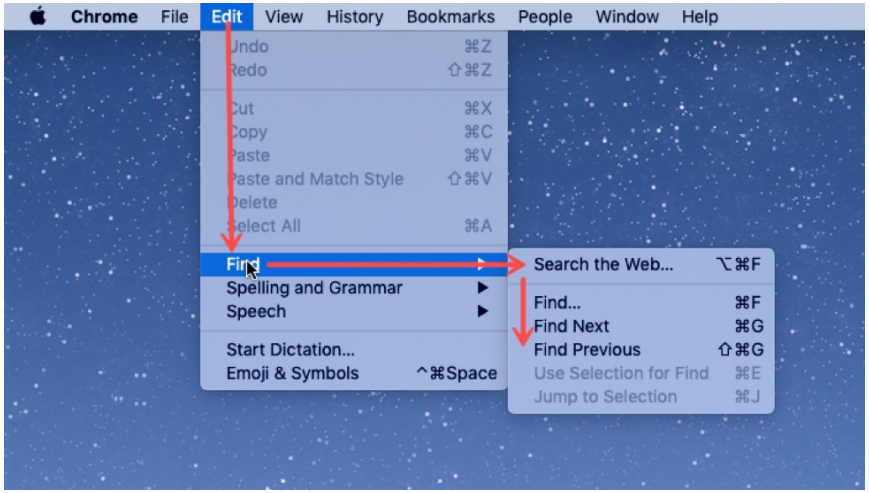
Example of steering law at work

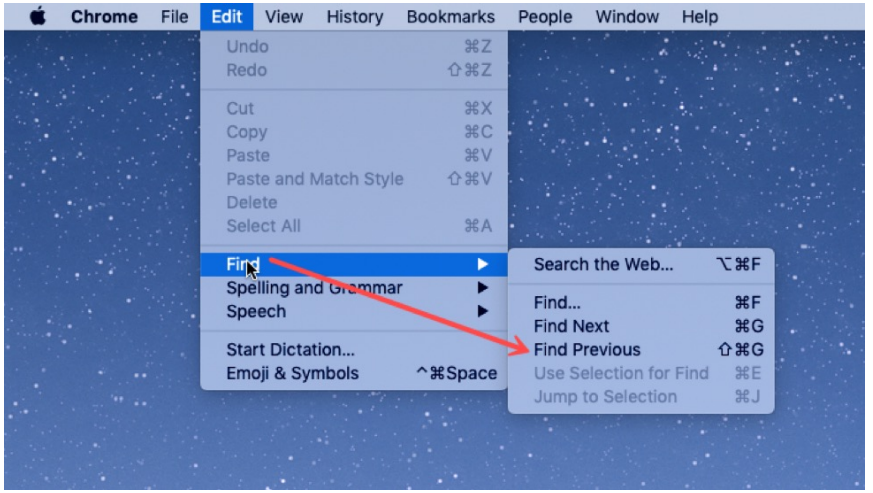
- **Sliders** and **scrollbars** allow you to move the pointer outside of the bar while you drag the handle
  - They track motion in the direction of the scrollbar only, and ignore any perpendicular motion
  - This effectively makes the constrained path as wide as the entire screen, greatly speeding scrollbar operation and eliminating a source of errors



Example of steering law at work

- **Hierarchical dropdown (or "pull-right") menus**
  - If the user moves the cursor outside the menu area, the menu disappears
  - That is why the width of the tunnel is important for how easily the user can steer through — a narrow tunnel makes it easy to accidentally exit the tunnel area



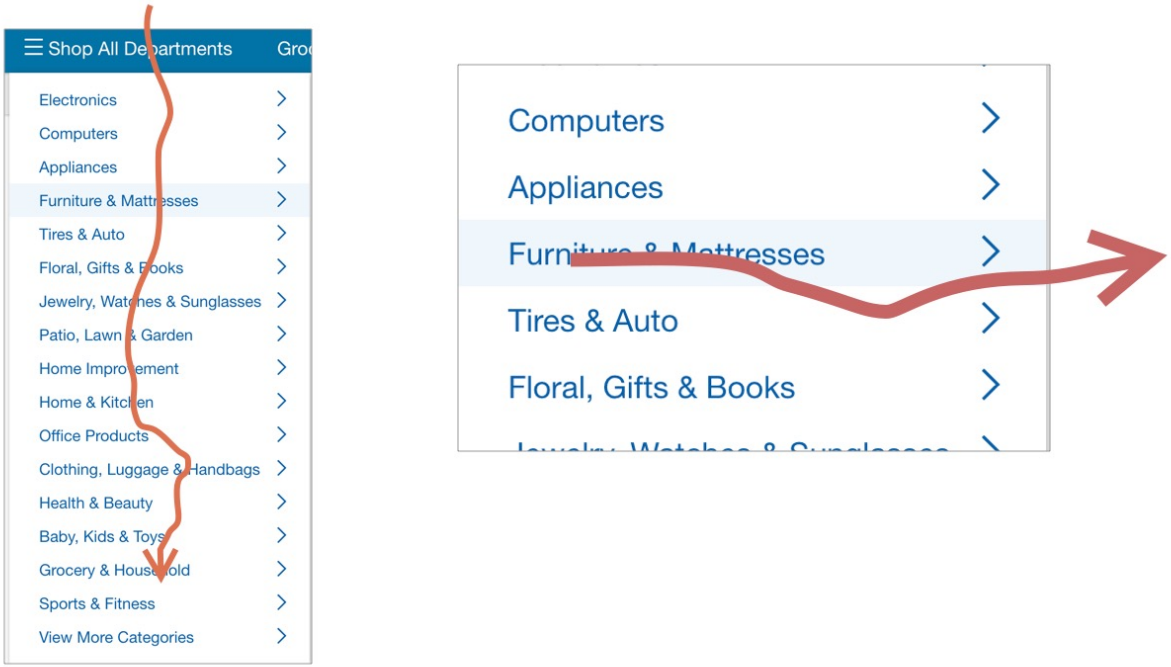


  - Many users will attempt to move diagonally from Find to an item in the child menu, but, because in doing so their mouse will cross the area for Spelling and Grammar, the target submenu will be lost and instead the Spelling and Grammar one will be opened



FIGURE 13.11 RoadScholar.org travel website: (A) in 2012, narrow menu items, and (B) in 2013, wider items.

… but often you need to make tradeoffs between vertical and horizontal optimizations



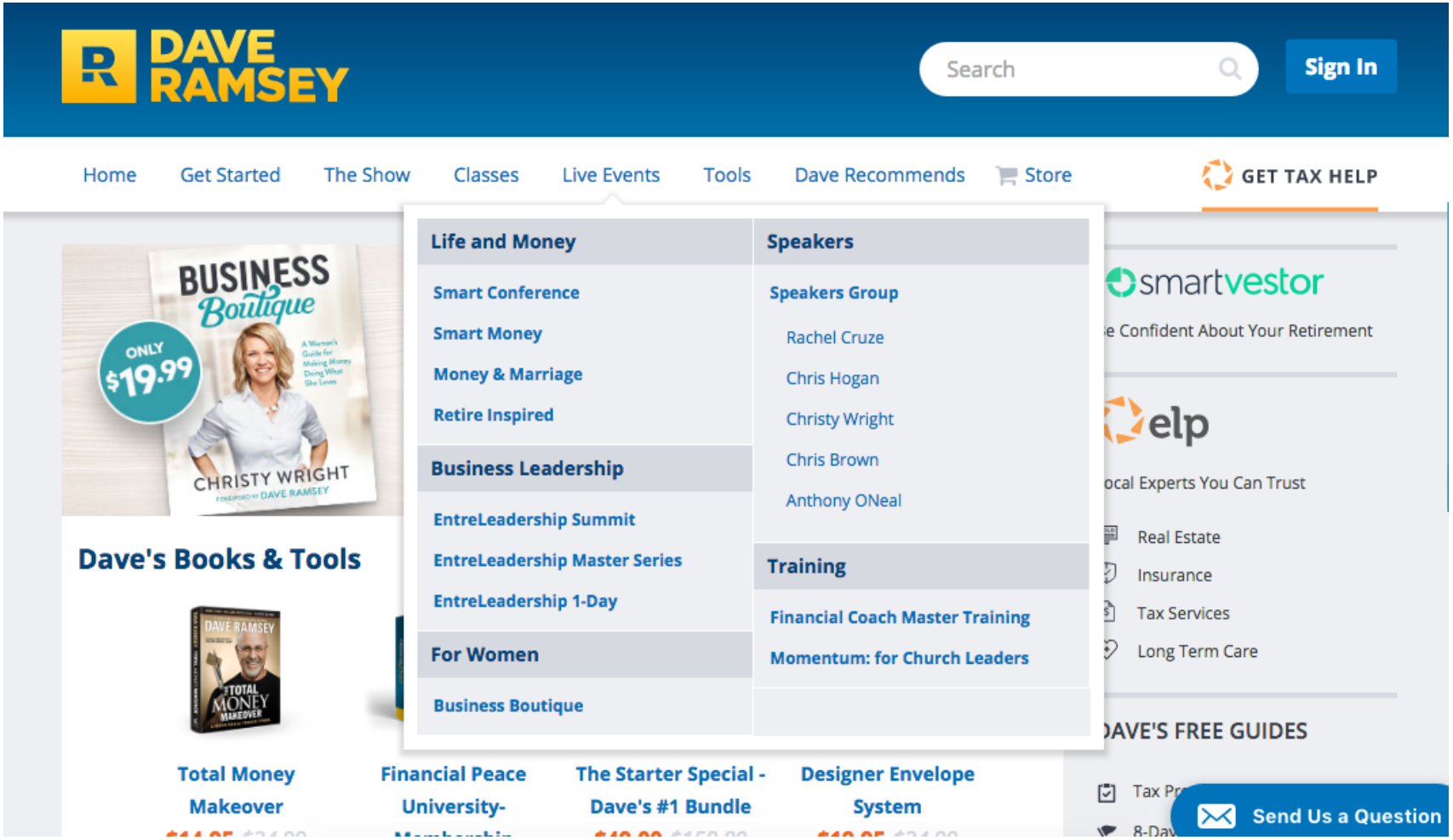
- This hierarchical menu shows the inherent **tradeoff** of optimizing a hierarchical menu for Steering Law effects and compromising between the sizes of the vertical tunnel (left) and the horizontal tunnel (right)
- A wide menu optimizes the **vertical** tunnel for ease of steering down the menu, especially for menus with many choices
- However, by making the menu wide, the length of the **horizontal tunnel** is increased, and moving across the menu to access the corresponding submenu is more fraught with errors

Design implications of the Steering law: Making Menus more Steer-Friendly

- **Keep dropdown menus as short as possible**
  - Menus with **a few choices** minimize the time and difficulty of steering through the narrow tunnel, and also reduces the time to visually search through a large number of choices
- **Avoid hierarchical menus**, particularly hierarchical menus that are more than two-levels deep. There will always be a **tradeoff**
  - The vertical tunnel needs to be short, but that means that each of the items in the menu may have to be given smaller height — which leads to narrow horizontal tunnels
  - The vertical tunnel must also be wide, but that leads to longer horizontal tunnels from main-menu option to the corresponding submenu

Mega Menus

- Consider **mega menus** as an alternative to pull-down menus
  - As mega menus allow for free movement within a wide 2D space, the steering law does not apply (though Fitts's Law still does, so make sure targets are still large)
- Mega menus are a type of expandable menus in which many choices are displayed in a two-dimensional dropdown layout
- They are an excellent design choice for accommodating a large number of options or for revealing lower-level site pages **at a glance**
- Characteristics
  - Big, two-dimensional panels divided into groups of navigation options
  - Navigation choices structured through layout, typography, and (sometimes) icons
  - Everything visible at once — no scrolling



The mega menu separated the content into five distinct groups. Users opening the mega menu could quickly scan the different categories

Mega menus beat regular dropdowns

- For bigger sites with many features, regular dropdown menus **typically hide most of the user's options**
  - Scrolling hides the options at the top of the menu.
  - As a result, you can't visually compare all your choices; **you have to rely on short-term memory**
  - Mega menus show everything at a glance, so users can see rather than try to remember
- While plain text can be wonderful, **illustrations** can indeed be worth a mouthful of words
  - Mega menus make it easy to use pictures and icons when appropriate
  - And, even if you stick to text alone, you have richer typography at your disposal (e.g., letting you differentiate link sizes according to their importance)

Design implications of the Steering law: Sliders and Scrollbars

- Sliders work **best when the specific value does not matter** to the user, but an approximate value is good enough
  - The wider or the denser the range selectable through a slider, the harder it is to select a precise value
  - Especially on touch interfaces, the level of control needed to meticulously operate a slider to an exact value is simply not realistic

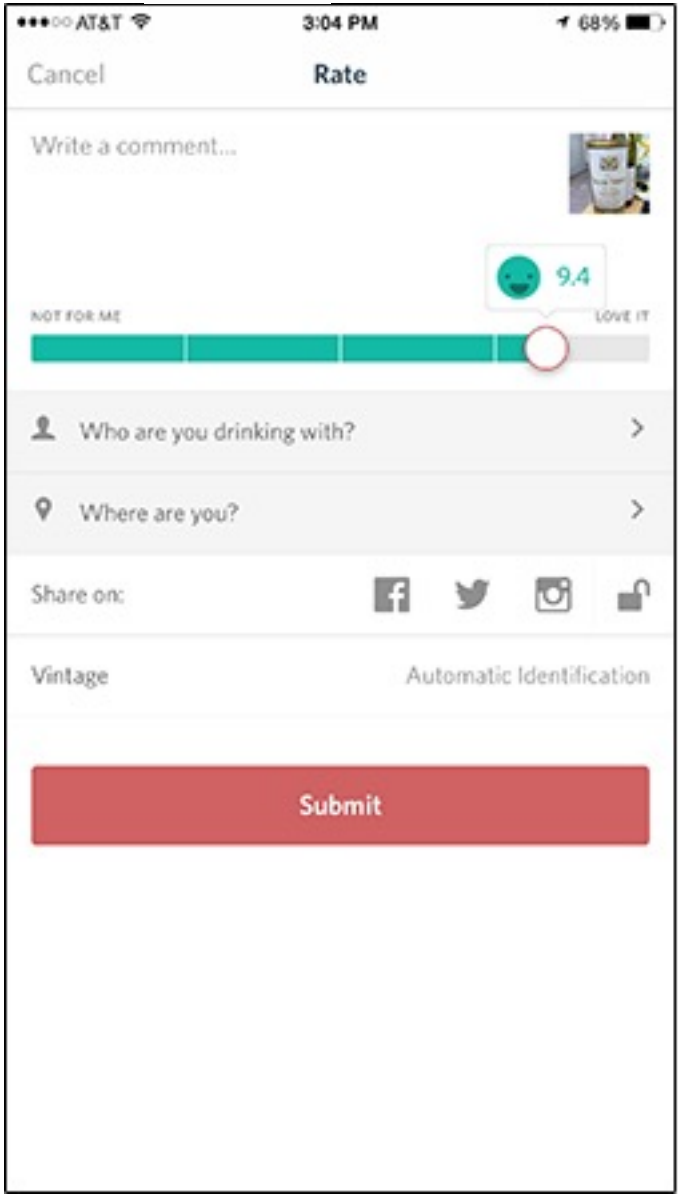
Example

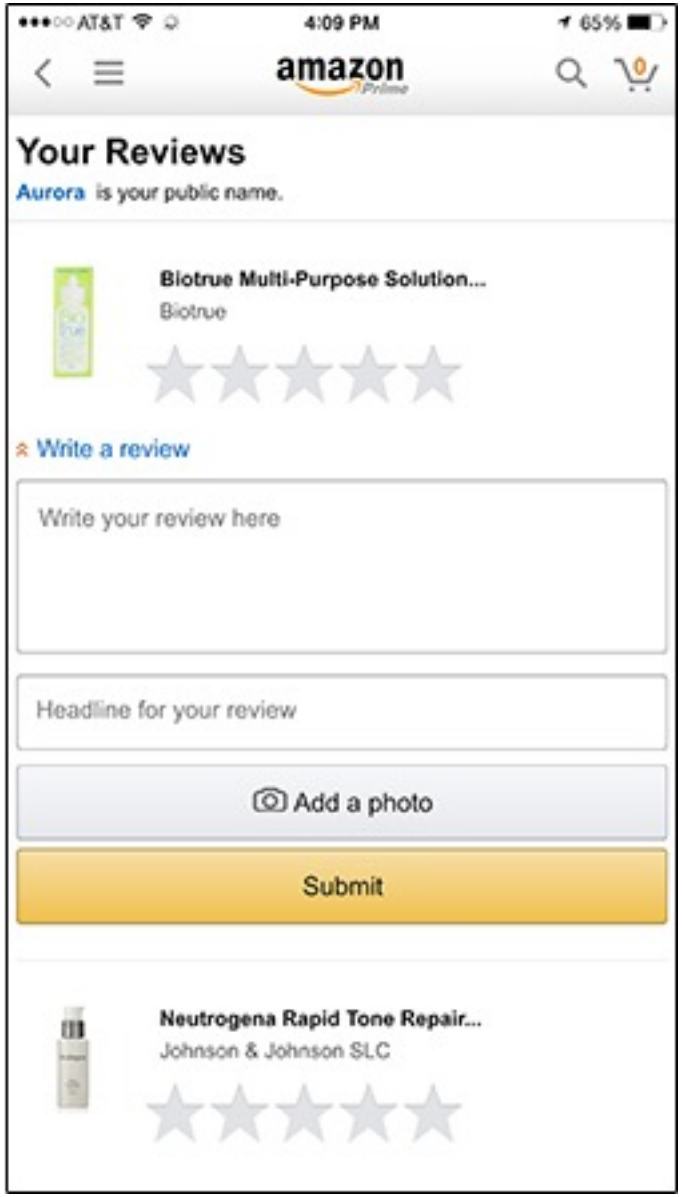


- This app uses a slider (that is an imprecise control) for something that needs to be indicated with precision and the slider moves in very small increments
- This is not very convenient



Examples





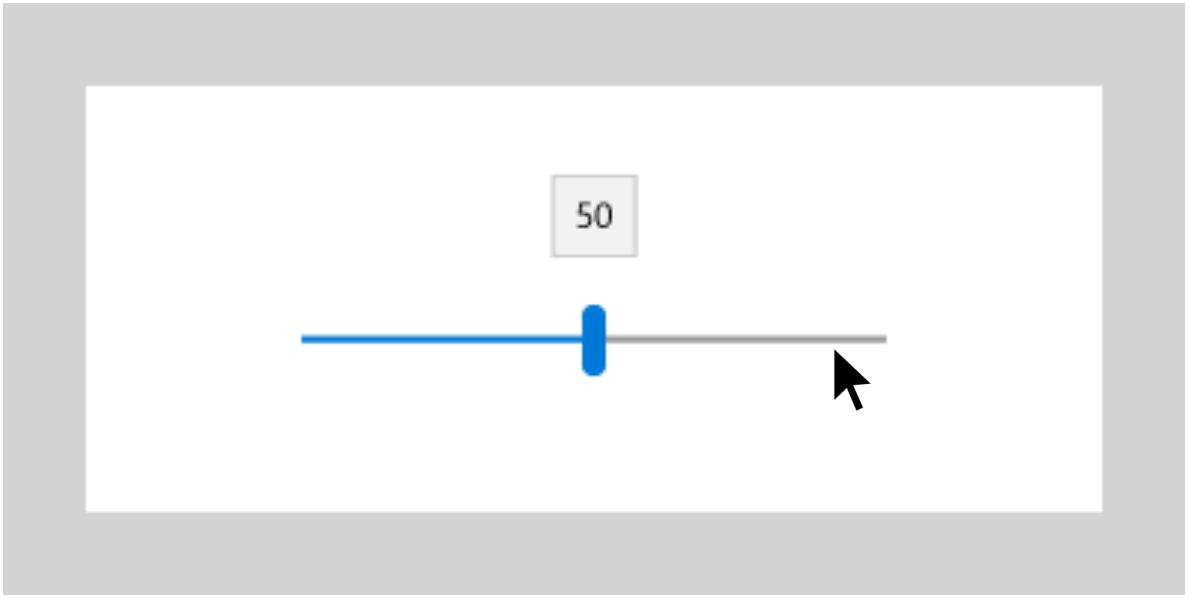
- Left: The Delectable.com app uses a slider for assigning a very specific rating to a bottle of wine
- Right: The Amazon.com app uses the more common, and easier to select, tappable stars to assign a rating for a product

Design implications of the Steering law: Sliders and Scrollbars

- When designing UI elements that involve path-steering tasks, such as sliders, scrollbars, and video playback heads**, remember that users will have a hard time achieving precision with such controls**
  - Therefore, for precise tasks, supplement these UI elements with other secondary controls that support precision
  - When using a slider to select a parameter value, **use the slider as a coarse control** (for reaching the general desired region), and provide a secondary fine control (such as numeric input box with stepper buttons) to choose a specific value

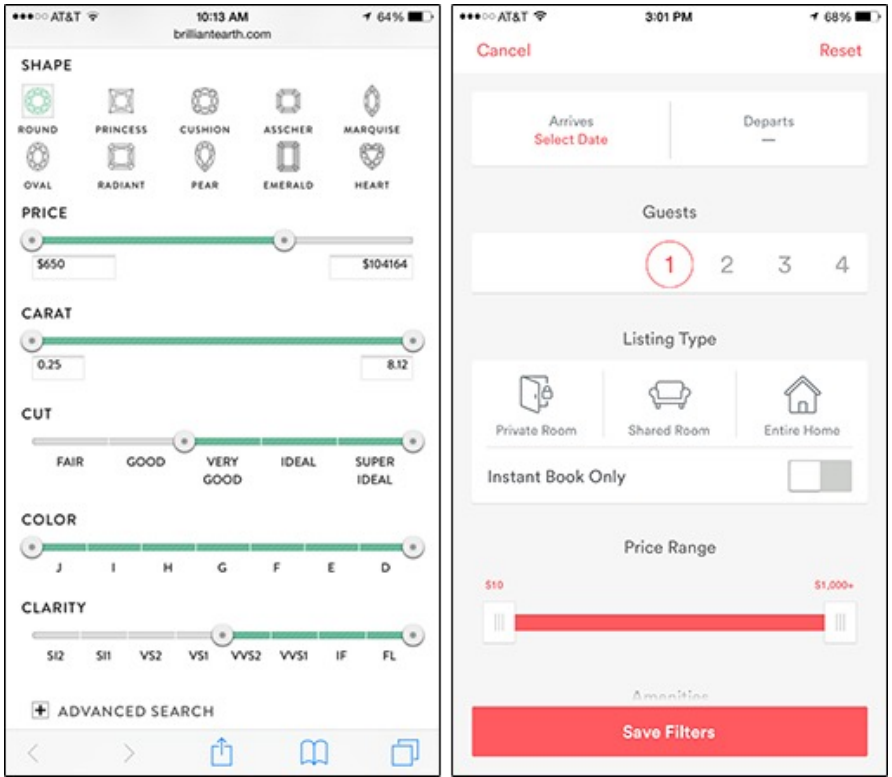
Design implications of the Steering law: Sliders and Scrollbars

- **Allow users to click anywhere on the slider to jump to the desired position**, rather than requiring them to click and drag
  - This approach enables a user to move freely in 2D space without needing to steer through the tunnel



Design implications of the Steering law: Sliders and Scrollbars

- On **touchscreens**, think about the overall target size of the slider knob and ensure that the user's finger **won't obscure the slider** (or any labels around it)
- To remain visible during use, any labels describing the slider or its currently selected value must appear **beside** or **above** the thumb of the user and the knob of the slider



Left: The sliders on BrilliantEarth.com place the labels describing the slider increments below the slider track, where they will be obscured by the user's thumb during the interaction

Right: AirBnB.com correctly displays the slider values above the UI element, so they would remain visible throughout use

We have time requirements

- To synchronize well with human users, interactive systems must meet certain real-time deadlines
- Interactive systems that don't synchronize well with users' time requirements are
  - Less effective
  - Users perceive them as unresponsive
- Perceived responsiveness, may seem less important than effectiveness, but in fact it is more important
- Researchers have found consistently that an interactive system's **responsiveness** (i.e., its ability to keep up with users, keep them informed about its status, and not make them wait unexpectedly) is **the most important factor** in determining user satisfaction, more so than ease of learning and ease of use

Responsiveness

- Responsive systems keep a user informed even if they cannot fulfill the user's requests immediately
- They provide feedback about what the user has done and what is happening, and they prioritize the feedback based on human perceptual, motor, and cognitive deadlines
- Specifically, they:
  - Let you know immediately that your input was received
  - Provide some indication of how long operations will take
  - Free you to do other things while waiting
  - Manage queued events intelligently
  - Perform housekeeping and low-priority tasks in the background
  - Anticipate your most common requests

Poor responsiveness

- Systems that display poor responsiveness
  - Do not meet human time deadlines
  - Don't keep up with users
  - Don't provide timely feedback for user actions, so users are unsure of what they have done or what the system is doing
  - Make users wait at unpredictable times and for unpredictable periods
  - Limit users' work pace – sometimes severely
- E.g.,
  - Delayed feedback for button presses, scrollbar movement, or object manipulations
  - Time-consuming operations that block other activity and cannot be aborted
  - Providing no clue how long lengthy operations will take
  - Jerky, hard-to-follow animations
  - Ignoring user input while performing "housekeeping" tasks users did not request

Guidelines to achieve responsiveness

- To be perceived as responsive, interactive systems should
  - Acknowledge user actions **instantly**, even if returning that the answer will take time; preserve users' perception of cause and effect
    - instantly means within about **100 ms**. Much longer than that, and the user interface will have moved out of the realm of cause and effect
  - Let users know when the software is busy and when it isn't
  - Free users to do other things while waiting for a function to finish
  - Animate movement smoothly and clearly
  - Allow users to abort (cancel) lengthy operations they don't want
  - Allow users to judge how much time lengthy operations will take
  - Do its best to let users set their own work pace

Progress bars examples

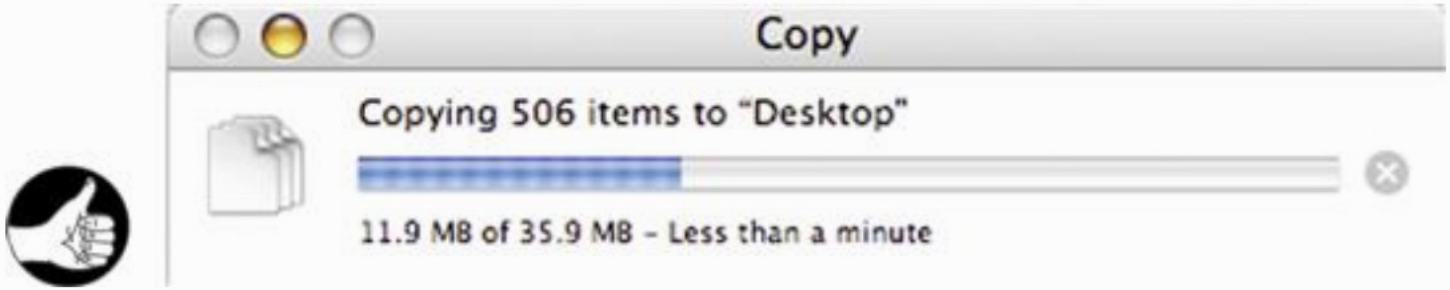


FIGURE 14.1 Mac OS X file transfer: good progress indicator, useful time estimate, and cancel button (circled X).

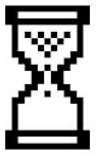


FIGURE 14.2 No progress bar (just a busy bar) and no cancel for (A) Mac OS X and (B) iMovie.

Busy indicators

- Software should display a busy indicator for any function that blocks further user actions while it is executing, even if the function normally executes quickly (e.g., in less than 0.1 second)
  - This indicator can be very helpful to a user if for some reason the function gets bogged down or hung up
  - It harms nothing: when the function executes at the normal speed, the busy indicator appears and disappears so quickly that users barely see it

- Static wait-cursors
  - They provide very little information: only that the software is temporarily occupied and unavailable to the user for other operations



static

- Animated wait-cursors
  - Wait animations are more "friendly" to users than static wait-cursors because they show that the system is working, not crashed or hung up

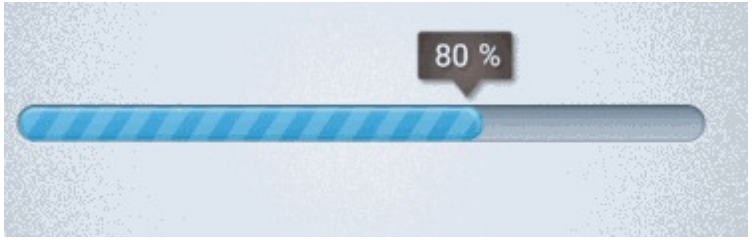


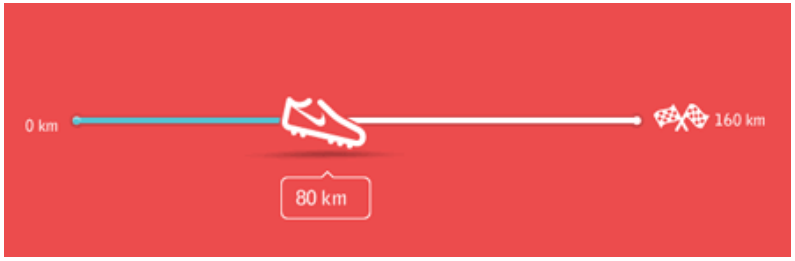
animation

Progress indicators

- Better than busy indicators because they let users see how much time remains
- Should be displayed for any operation that will take longer than a few seconds







- Get inspiration:

https://www.justinmind.com/blog/10-inspiring-progress-bars-that-delight-users/

https://www.mockplus.com/blog/post/progress-bar-design

Progress indicators: guidelines

- **Show work remaining, not work completed**
  - Bad: "3 files copied." Good: "3 of 4 files copied."
- **Show total progress, not progress on the current step**
  - Bad: "5 seconds left on this step." Good: "15 seconds left."
- To show the percentage of an operation that is complete, **start at 1%, not 0%**
  - Users worry if the bar stays at 0% for more than 1–2 seconds
- **Display 100% only very briefly at the end of an operation**
  - If the progress bar stays at 100% for more than 1–2 seconds, users assume it's wrong
- **Show smooth, linear progress, not erratic bursts of progress**
- **Use human-scale precision, not computer precision**
  - Bad: "240 seconds." Good: "About 4 minutes"

Lesson for the course “Human-Computer Interaction”

Google Material Design (Part 3)

Prof. Luca Turchet

UNIVERSITY OF TRENTO - Italy

Cards

- https://material.io/components/cards/

Lists

- https://material.io/components/lists/

Confirmation and acknowledgment

- https://material.io/design/communication/confirmation-acknowledgement.html

Help and feedback

- https://material.io/design/communication/help-feedback.html

Imagery

- https://material.io/design/communication/imagery.html

Writing

- https://material.io/design/communication/writing.html

Homework

- Select a software program, a website and an app and analyze them in terms of the Fitts' law implications for design
- Idem for the Steering law
- Analyze a website under the lens of responsiveness
- Choose one of the Material Design case studies and have a look at it:
  - https://material.io/design/material-studies/about-our-material-studies.html

Bibliography

- Chapters 13 and 14 of the book *"Designing with the Mind in Mind: Simple Guide to Understanding User Interface Design Guidelines"* - 2nd Edition - by Jeff Johnson
- https://www.interaction-design.org/literature/article/fitts-s-law-the-importance-of-size-and-distance-in-ui-design
- https://www.smashingmagazine.com/2012/12/fittss-law-and-user-experience/
- https://blog.hubspot.com/marketing/fitts-law
- https://www.nngroup.com/articles/steering-law/
- https://www.nngroup.com/articles/mega-menus-work-well/
- https://www.nngroup.com/articles/gui-slider-controls/
- https://www.nngroup.com/articles/touch-target-size/
- https://iconshots.com/technology/applying-fittss-law-to-mobile-user-experience-how-does-it-work/